

Análisis

P00 Sección 30

Daniel Sacol - 26870

Tabla de atributos

Clase	Atributo	Tipo	Visibilidad
Orden	numeroOrden	int	-
Orden	nombrePropietario	String	-
Orden	placa	String	-
Orden	descripcionServicio	String	-
Orden	costoEstimado	double	-
ControladorOrdenes	ordenes	List<Orden>	-
VistaOrdenes	controlador	ControladorOrdenes	-
VistaOrdenes	scanner	Scanner	-

Tabla de métodos

Clase	Método	Visibilidad	Parámetros	Retorno
Orden	Orden (Constructor)	+	int numeroOrden, String nombrePropietario, String placa, String descripcionServicio, double costoEstimado	N/A
Orden	getNumeroOrden	+	Ninguno	int
Orden	getNombrePropietario	+	Ninguno	String
Orden	getPlaca	+	Ninguno	String
Orden	getDescripcionServicio	+	Ninguno	String
Orden	getCostoEstimado	+	Ninguno	double
Orden	setDescripcionServicio	+	String descripcionServicio	void
Orden	setCostoEstimado	+	double costoEstimado	void
Orden	toString	+	Ninguno	String
ControladorOrdenes	ControladorOrdenes (Constructor)	+	Ninguno	N/A

ControladorOrdenes	registrarOrden	+	Orden nuevaOrden	void
ControladorOrdenes	consultarOrdenes	+	Ninguno	ArrayList<>(ordenes)
ControladorOrdenes	buscarOrden	+	int numOrden	Orden
ControladorOrdenes	modificarOrden	+	int numOrden, String nuevaDescripcion, double nuevoPrecio	void
ControladorOrdenes	cancelarOrden	+	int numOrden	void
ControladorOrdenes	consultarOrdenesPorPlaca	+	String placa	List<orden>
ControladorOrdenes	calcularTotalOrdenes	+	Ninguno	double
ControladorOrdenes	calcularCostoPromedio	+	Ninguno	double
ControladorOrdenes	obtenerOrdenMayorCosto	+	Ninguno	Orden
ControladorOrdenes	obtenerCantidadOrdenes	+	Ninguno	int
VistaOrdenes	VistaOrdenes (Constructor)	+	Ninguno	N/A
VistaOrdenes	mostrarMenu	+	Ninguno	void
VistaOrdenes	leerNuevaOrden	+	Ninguno	Orden
VistaOrdenes	mostrarListaOrdenes	+	List<orden> lista	void
VistaOrdenes	mostrarMensaje	+	String mensaje	void
VistaOrdenes	imprimirOpcionesMenu	+	Ninguno	void
VistaOrdenes	ejecutarRegistroOrden	+	Ninguno	void
VistaOrdenes	ejecutarConsultaTodas	+	Ninguno	void
VistaOrdenes	ejecutarBusquedaPorNumero	+	Ninguno	void
VistaOrdenes	ejecutarBusquedaPorPlaca	+	Ninguno	void
VistaOrdenes	ejecutarModificacionOrden	+	Ninguno	void
VistaOrdenes	ejecutarCancelacionOrden	+	Ninguno	void
VistaOrdenes	ejecutarCalculoTotal	+	Ninguno	void
VistaOrdenes	ejecutarObtenerMayorCosto	+	Ninguno	void
VistaOrdenes	leerNuevaOrden	+	Ninguno	Orden
VistaOrdenes	mostrarListaOrdenes	+	lista	void
Main	main	+	String[] args	void

4. ¿Qué información deberá almacenarse dentro de la colección dinámica?

Objetos de tipo Orden

5. ¿Dónde utilizará List y dónde realizará la creación mediante ArrayList?

En la clase ControladorOrdenes, tendrá un atributo List y en su constructor se inicializa como ArrayList.

8. ¿Cómo proveerá de valores iniciales a sus objetos? ¿Qué valores iniciales les asignará?

Mediante su método Constructor, se le asigna los valores que se le pasen a este método.

9. ¿Cómo identificará una orden específica dentro de la colección?

Mediante el numero de orden se buscará el objeto.

10. ¿Cómo agregará y eliminará órdenes de la colección dinámica?

Para agregar, se instanciará un objeto de tipo Orden y se insertará en la lista mediante el método .add() de la interfaz List. Para eliminar, primero se buscará el objeto que coincida con el número de orden y posteriormente se eliminará de la lista utilizando el método. remove()

11. ¿Cómo realizará la búsqueda de todas las órdenes asociadas a una misma placa?

Se buscan los objetos que coincidan con la placa y se agregan a una lista.

12. ¿Cómo recorrerá la colección para calcular el valor total y el costo promedio de las órdenes?

Mediante un ciclo for, se acumulará en una variable auxiliar el valor de costo estimado de cada objeto en la lista, el cual también será útil para dividir entre la cantidad de objetos o el tamaño de la lista.

13. ¿Cómo determinará cuál es la orden con el costo estimado más alto?

Comparando el valor del costo de cada objeto dentro de la lista.

14. ¿Qué situaciones dentro del programa pueden producir una excepción?

- Buscar una orden que no existe en la lista por su numero de orden
- Buscar las ordenes de una placa que no existe
- Cálculos como el promedio sobre una lista vacía
- Entradas de enteros y cadenas en las vistas.

15. ¿Qué operaciones serán protegidas utilizando try-catch?

- La navegación del menú principal
- La lectura de datos de teclado
- El registro y modificación de órdenes
- Las operaciones de búsqueda, modificación y cancelación
- Los cálculos matemáticos (Promedio y Mayor Costo)

16. ¿Qué acción realizará mediante el bloque finally y por qué debe ejecutarse independientemente del resultado de la operación?

En la clase VistaOrdenes, el bloque finally se utiliza para imprimir una línea separadora visual en la consola. Debe ejecutarse siempre porque garantiza que la interfaz de usuario de la consola se mantenga limpia, ordenada y legible.